

DSA0503 – Query Processing for Data Science

LAB EXPERIMENTS (11-20)

Divyesh S P (192424147)

Experiment 11

Aim

To create a DataFrame with random values, convert some values into NaN, and highlight the NaN values using Pandas styling.

Algorithm

1. Import the required libraries.
2. Create a DataFrame with random values.
3. Replace selected values with NaN.
4. Apply conditional formatting to highlight NaN values.
5. Display the styled DataFrame.

Output

	A	B	C	D
0	10.000000	16.000000	65.000000	29.000000
1	90.000000	94.000000	nan	9.000000
2	74.000000	1.000000	41.000000	37.000000
3	nan	12.000000	55.000000	89.000000
4	63.000000	34.000000	73.000000	79.000000
5	50.000000	nan	55.000000	78.000000
6	70.000000	14.000000	26.000000	14.000000
7	93.000000	87.000000	31.000000	nan
8	90.000000	13.000000	66.000000	32.000000
9	58.000000	37.000000	28.000000	19.000000

Result

The DataFrame was successfully created, and the NaN values were highlighted.

Experiment 12

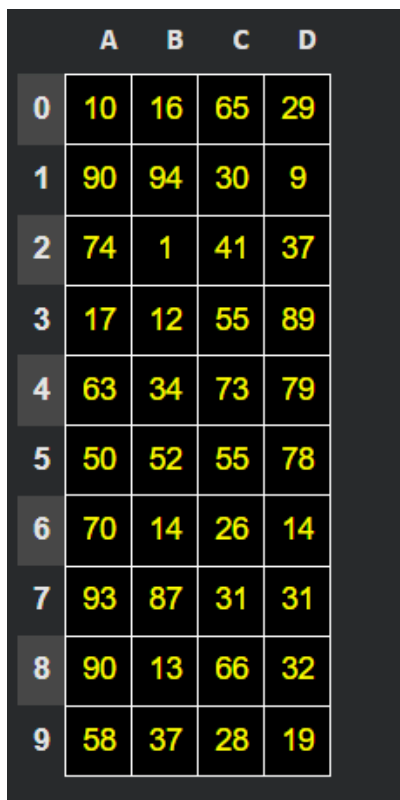
Aim

To create a DataFrame with random values and set the DataFrame background color to black and font color to yellow using Pandas styling.

Algorithm

1. Import the required libraries.
2. Create a DataFrame with random values.
3. Apply background color as black.
4. Apply font color as yellow.
5. Display the styled DataFrame.

Output



	A	B	C	D
0	10	16	65	29
1	90	94	30	9
2	74	1	41	37
3	17	12	55	89
4	63	34	73	79
5	50	52	55	78
6	70	14	26	14
7	93	87	31	31
8	90	13	66	32
9	58	37	28	19

Result

The DataFrame was successfully styled with the specified background and font colors.

Experiment 13

Aim

To detect the missing values in a given DataFrame and display the result as True or False.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Detect the missing values.
4. Display True for missing values and False for available values.

Output

Original DataFrame:

	Name	Age	Marks	City
0	Alice	25.0	85.0	Chennai
1	Bob	NaN	90.0	Delhi
2	Charlie	30.0	NaN	NaN
3	David	28.0	75.0	Mumbai
4	Eva	NaN	88.0	Bangalore

Missing Values (True = Missing, False = Not Missing):

	Name	Age	Marks	City
0	False	False	False	False
1	False	True	False	False
2	False	False	True	True
3	False	False	False	False
4	False	True	False	False

Result

The missing values in the DataFrame were successfully detected and displayed.

Experiment 14

Aim

To find and replace the missing values in a given DataFrame using Pandas.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Identify the missing values.
4. Replace the missing values with suitable values.
5. Display the updated DataFrame.

Output

Original DataFrame:

	Name	Age	Marks	City
0	Alice	25.0	85.0	Chennai
1	Bob	NaN	90.0	NaN
2	Charlie	30.0	NaN	Mumbai
3	David	NaN	75.0	Delhi
4	Eva	28.0	NaN	NaN

DataFrame after Replacing Missing Values:

	Name	Age	Marks	City
0	Alice	25.000000	85.0	Chennai
1	Bob	27.666667	90.0	Unknown
2	Charlie	30.000000	0.0	Mumbai
3	David	27.666667	75.0	Delhi
4	Eva	28.000000	0.0	Unknown

Result

The missing values were successfully replaced in the DataFrame.

Experiment 15

Aim

To keep only the rows containing at least two NaN values in a given DataFrame.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Count the NaN values in each row.
4. Filter the rows containing at least two NaN values.
5. Display the filtered DataFrame.

Output

Original DataFrame:

	Name	Age	Marks	City
0	Alice	25.0	85.0	Chennai
1	Bob	NaN	NaN	Delhi
2	Charlie	NaN	NaN	NaN
3	David	28.0	75.0	NaN
4	Eva	NaN	88.0	NaN

Rows with at least 2 NaN values:

	Name	Age	Marks	City
1	Bob	NaN	NaN	Delhi
2	Charlie	NaN	NaN	NaN
4	Eva	NaN	88.0	NaN

Result

The rows with at least two NaN values were successfully filtered and displayed.

Experiment 16

Aim

To split the DataFrame into groups based on the School Code and check the type of the GroupBy object.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Group the data using the School Code.
4. Check the type of the GroupBy object.
5. Display the grouped data.

Output

Students Grouped by School Code:

School Code: S001

	School_Code	Student_Name	Age	Class
0	S001	John	15	10
1	S001	Alice	16	10
6	S001	Daniel	14	8
9	S001	Lily	14	9

School Code: S002

	School_Code	Student_Name	Age	Class
2	S002	David	15	9
3	S002	Sophia	14	9
7	S002	Olivia	15	10

School Code: S003

	School_Code	Student_Name	Age	Class
4	S003	Michael	16	10
5	S003	Emma	15	9
8	S003	James	16	8

Result

The DataFrame was successfully grouped based on the School Code, and the GroupBy object type was displayed.

Experiment 17

Aim

To split the DataFrame based on School Code and calculate the mean, minimum, and maximum age for each school.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Group the data using the School Code.
4. Calculate the mean, minimum, and maximum age.
5. Display the results.

Output

Original DataFrame:

	School_Code	Student_Name	Age	Class
0	S001	John	15	10
1	S001	Alice	16	10
2	S002	David	15	9
3	S002	Sophia	14	9
4	S003	Michael	16	10
5	S003	Emma	15	9
6	S001	Daniel	14	8
7	S002	Olivia	15	10
8	S003	James	16	8
9	S001	Lily	14	9

Mean, Minimum and Maximum Age for each School:

	mean	min	max
School_Code			
S001	14.750000	14	16
S002	14.666667	14	15
S003	15.666667	15	16

Result

The statistical values for age were successfully calculated and displayed for each school.

Experiment 18

Aim

To split the DataFrame into groups based on School Code and Class.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Group the data using School Code and Class.
4. Display each group.
5. Verify the grouped output.

Output

```
Groups based on School Code and Class:
School Code: S001, Class: 8
  School_Code Student_Name  Age  Class
6         S001      Daniel   14      8
-----
School Code: S001, Class: 9
  School_Code Student_Name  Age  Class
9         S001       Lily   14      9
-----
School Code: S001, Class: 10
  School_Code Student_Name  Age  Class
0         S001       John   15     10
1         S001      Alice   16     10
-----
School Code: S002, Class: 9
  School_Code Student_Name  Age  Class
2         S002      David   15      9
3         S002     Sophia   14      9
-----
School Code: S002, Class: 10
  School_Code Student_Name  Age  Class
7         S002     Olivia   15     10
-----
School Code: S003, Class: 8
  School_Code Student_Name  Age  Class
8         S003      James   16      8
-----
School Code: S003, Class: 9
  School_Code Student_Name  Age  Class
5         S003      Emma   15      9
-----
School Code: S003, Class: 10
  School_Code Student_Name  Age  Class
4         S003    Michael   16     10
-----
```

Result

The DataFrame was successfully grouped according to School Code and Class.

Experiment 19

Aim

To display the dimensions (shape) of the World Alcohol Consumption dataset and extract the column names.

Algorithm

1. Import the Pandas library.
2. Read the World Alcohol Consumption dataset.
3. Find the dimensions of the dataset.
4. Extract the column names.
5. Display the results.

Output

```
World Alcohol Consumption Dataset:

   country  year  spirit_servings  wine_servings  beer_servings
0  Afghanistan  1989           0           0           0
1    Albania  1989          132           54           89
2    Algeria  1989           0           14           25
3   Andorra  1989          312          312          245
4    Angola  1989           35            1          217

Shape of the DataFrame:
(5, 5)

Column Names:
Index(['country', 'year', 'spirit_servings', 'wine_servings', 'beer_servings'], dtype='object')
```

Result

The dimensions and column names of the dataset were successfully displayed.

Experiment 20

Aim

To find the index of a given substring in a DataFrame column using Pandas.

Algorithm

1. Import the Pandas library.
2. Create or read the DataFrame.
3. Specify the substring to search.
4. Find the index of the substring in the required column.
5. Display the index values.

Output

```
Original DataFrame:
```

```
      Name
0  John Smith
1 Alice Johnson
2 David Brown
3 Sophia Williams
4 Michael Taylor
```

```
DataFrame with Substring Index:
```

```
      Name  Substring_Index
0  John Smith             -1
1 Alice Johnson            10
2 David Brown             -1
3 Sophia Williams         -1
4 Michael Taylor          -1
```

Result

The index of the given substring was successfully identified and displayed.